МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждение образования
«Витебский государственный технологический университет»

ТЕСТИРОВАНИЕ И ОТЛАДКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Методические указания по выполнению
лабораторных работ для студентов специальностей
6-05-0611-01 «Информационные системы и технологии
(в проектировании и производстве)»
6-05-0611-04 «Информационные системы и технологии
(в бизнес-менеджменте)»

2025 г.

УДК 004.9
ББК 32.97
Р17

Составители:

А. С. Соколова, Д. В. Черненко

Одобрено кафедрой «Информационные системы и технологии»
УО «ВГТУ», протокол №12 от 18.04.2025.

Рекомендовано к изданию редакционно-издательским
советом УО «ВГТУ», протокол №__ от __.__.2025.

Тестирование и отладка программного обеспечения : методические указания по выполнению лабораторных работ / УО «ВГТУ» ; сост.: А. С. Соколова, Д. В. Черненко – Витебск, 2025. – 32 с.

Методические указания предназначены для проведения лабораторных работ связанных с вопросами анализа, планирования, проведения тестовых испытаний и оценки качества программного обеспечения на всех стадиях его жизненного цикла Основной упор сделан на процессы модульного и интеграционного тестирования, их этапы, возможности автоматизации, особенности работы с инструментами автоматизации.

УДК 004.9
ББК 32.97

© УО «ВГТУ», 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4

ЛАБОРАТОРНАЯ РАБОТА №1 Тест-план, чек-лист и тест-кейс 5

ЛАБОРАТОРНАЯ РАБОТА №2 Модульное тестирование, использование JUnit 9

ЛАБОРАТОРНАЯ РАБОТА №3 Разработка проекта на Java под управлением тестированием 11

ЛАБОРАТОРНАЯ РАБОТА №4 Тестирование внешних зависимостей, использование jMock. 15

ЛАБОРАТОРНАЯ РАБОТА №5 Тестирование web-приложений, Selenium IDE 19

ЛАБОРАТОРНАЯ РАБОТА №6 Автоматизированное тестирование Selenium WebDriver на языке Java 22

ЛАБОРАТОРНАЯ РАБОТА №7 Автоматизированное тестирование Selenium WebDriver на языке C# 26

ЛАБОРАТОРНАЯ РАБОТА №8 Поиск и документирование дефектов 28

ЛИТЕРАТУРА 32

ВВЕДЕНИЕ

Настоящее методическое указание направлено на получение студентами практических знаний и навыков по современным технологиям тестирования программного обеспечения и включает как ручное, так и автоматизированное тестирование, необходимое для специальности «Информационные системы и технологии». В процессе выполнения лабораторных работ предполагается практически ознакомиться с такими разделами как:

- планирование тестирования;
- разработка тестов;
- реализация автоматизированных тестов;
- поиск и описание дефектов;

В результате выполнения лабораторных работ студент не только знакомиться с основами тестирования, но и получает практические навыки, которые необходимо использовать при разработке информационных систем, реализуемых на дипломном проектировании. Поэтому при выполнении лабораторных работ необходимо подробно документировать все шаги выполнения заданий в виде отчетов, с детальным описанием выполняемых действий.

Отчет по лабораторным работам должен содержать:

- Название лабораторной работы
- ФИО студента и дату выполнения работы;
- Цель работы
- Подробное описание действий и результатов с использованием скриншотов;
- Выводы.

В выводах необходимо отразить всё новое, что вы узнали, выполняя лабораторную работу. Например, что главная цель тестирования, это обеспечение качества путем удовлетворения потребностей заказчика, а не поиск ошибок. Перечислять что вы сделали в лабораторной работе в выводах нет необходимости.

В методических указаниях объем теоретического материала сведен к минимуму, а основной упор сделан на практические примеры. Там, где необходимо теоретическая информация даны ссылки на литературу с указанием страниц.

В качестве задания для лабораторных работ можно использовать тему, предполагаемую для дипломного проектирования. В случае если она ещё не сформулирована, то тема проекта для тестирования будет определен преподавателем.

ЛАБОРАТОРНАЯ РАБОТА №1 Тест-план, чек-лист и тест-кейс

Цель: познакомиться с основными видами тестовой документации:

- План тестирования (Test Plan);
- Чек-лист (Check-List);
- Тест-кейс (Test Case);

Краткие теоретические сведения

План тестирования – это документ, который описывает объем работ по тестированию проекта. Он описывает стратегию тестирования, цели, график, оценку, результаты, а также ресурсы необходимые для тестирования и составляется до начала тестирования.

Форм-фактор у тест-плана может быть разный (схема, интеллектуальная карта и т.д.). Мы будем составлять тест-план в виде документа.

Тест-план должен содержать следующие пункты[1]:

- Описание объекта тестирования: системы, приложения, инструмент разработки.
- Список функций и описание тестируемой системы и её компонентов в отдельности.
- Стратегия тестирования, а именно: виды тестирования и их применение по отношению к объекту тестирования.
- Последовательность проведения работ: подготовка, тестирование, анализ результатов в разрезе запланированных фаз разработки.
- Критерии начала тестирования — это готовность тестируемого проекта, законченность разработки требуемого функционала, наличие всей необходимой документации и т.д.
- Критерии окончания тестирования, когда результаты тестирования удовлетворяют критериям качества продукта: требования к количеству открытых багов, тестовое покрытие[2], использование тестов с разным уровнем детализации (модульное, интеграционное, системное) и т.д.

В качестве примера приведем пример составления тест-плана для проекта «Калькулятор».

Введение

Целью составления данного Тест Плана является описание процесса тестирования проекта «Калькулятор».

Объект тестирования

Библиотека calc.jar разработанная на языке Java, web-интерфейс для калькулятора.

Предполагаемые проверки:

- Модульное тестирование

[1] Куликов С.С. Тестирование программного обеспечения Базовый курс Раздел 2.6.2. «Тест-план и отчёт о результатах тестирования» стр.198

[2] Отношение атомарных методов, проверяемых хотя бы одним тестом к общему количеству методов, выраженное в процентах.

- Функциональное тестирование;
- Тестирование пользовательского интерфейса.

Тестируемый функционал:

- Дымовое тестирование;
- Ввод данных;
- Операции;
- Специальные тесты;
- Завершение работы;

Нагрузочное тестирование и тестирование безопасности проводиться не будет в виду отсутствия необходимых ресурсов.

Процесс тестирования:

Для максимального покрытия проекта предполагается составление чек-листов и тест-кейсов.

Автоматизация тестирования

При написании кода будет использоваться модульное тестирование с помощью фреймворка JUnit

На этапе интеграционного тестирования будет использован инструмент линейки Selenium. Выбор конкретного инструмента будет произведен на основании наличия необходимых ресурсов.

Критерии начала и окончания тестирования:

Тестирование может быть начато, если выполнены следующие условия:

1. Готова и утверждена необходимая документация;
2. Тестируемый функционал окончен и готов для передачи в тестирование.

Тестирование окончено, если выполнены следующие условия:

1. Все найденные дефекты задокументированы.

План работ

Задача	Время	Дата начала	Дата окончания
Составление тест плана и чек-листа	4 час	01.04.2023	03.04.2023
Корректировка тест плана и чек-листа	1 час	04.04.2023	02.04.2023
Написание модульных тестов [3]	5 % от времени разработки	03.04.2023	26.04.2023
Разработка интеграционных тестов	6 часов	27.04.2023	27.04.2023
Выполнение интеграционных	0,5 часа	28.04.2023	28.04.2023

[3] В процессе разработки кода

тестов			
Написание баг-репорта	2 часа	28.04.2023	28.04.2023

Чек-лист - документ, описывающий, что должно быть протестировано в проекте. По сути, это набор идей, которые можно сформировать либо на основе опыта работы над подобными проектами, либо на основе поиска информации в интернете, либо на основании коллективного мозгового штурма.

Разработка чек-листа проводится на этапе, когда сформированы пользовательские и системные требования к разрабатываемому продукту, но ещё до начала работы над кодом проекта, однако может корректироваться и на этапе разработки и отладки.

Чек-лист необходим чтобы:

- не забыть, что нужно протестировать;
- оптимизировать и упорядочить тестирование;
- структурировать последовательность процесса тестирования;

Пример чек-листа для тестирования десктопного приложения «Калькулятор» составлен в виде таблицы и приведен в отдельном документе[4]. Данный чек-лист достаточно подробный. Это хорошо для реального проекта и способствует обеспечению высокого качества[5]. Для дипломного проекта, сложность которого в разы больше, чем приведенное приложение детальность чек-листа может быть уменьшена. Однако это не означает ограничение фокуса тестирования, на какой-то отдельной функциональности. **Тесты должны охватывать все модули проекта!**

Тест-кейс – это чёткое описание действий, которые необходимо выполнить, для того чтобы проверить работу программы (поля для ввода, кнопки и т.д.). Данное описание содержит: действия, которые надо выполнить до начала проверки — предусловия; действия, которые надо выполнить для проверки — шаги; описание того, что должно произойти, после выполнения действий для проверки — ожидаемый результат.

Один тест-кейс соответствует одному пункту чек-листа. Но это скорее исключение, чем правило. Чек лист — это набор идей, а тест-кейс — это непосредственная реализация идеи и вариантов её использования намного больше. Поэтому одну идею чек-листа нужно максимально покрыть тестами (оформленных в виде тест-кейсов). Поскольку исчерпывающее тестирование невозможно, то необходимо используя теорию тестирования покрыть минимальным количеством тестов максимальный объем разрабатываемого продукта. Для этого используют техники тест-дизайна.

[4]Литература\ChekListCalc

[5] Под качеством программного продукта понимают удовлетворение требований клиента и степень готовности продукта

Наиболее полный формат тест-кейса описан в книге Куликова [1] стр. 117 в разделе 2.4.3. «Атрибуты (поля) тест-кейса». Он состоит из следивших пунктов:

- Идентификатор;
- Приоритет;
- Связанное с тест-кейсом требование;
- Модуль и подмодуль приложения;
- Заглавие (суть) тест-кейса;
- Исходные данные, приготовления;
- Шаги тест-кейса и ожидаемые результаты;

Пример тест-кейсов для тестирования нескольких функций приложения «Калькулятор» составлен в виде таблицы и приведен в отдельном документе [6]. С примером также можно ознакомиться в рабочей тетради «Автоматизированное тестирование» [2] раздел 1.4.3. «Создание и оформление тест-кейсов» стр. 31 и раздел 1.4.4. «Свойства качественных тест-кейсов» стр. 33.

Содержание отчета

Разработать для тестируемого проекта тестовую документацию:

- План тестирования;
- Чек-лист;
- Четыре тест-кейса;

В качестве задания для лабораторных работ лучше всего взять тему, предполагаемую для дипломного проектирования. В случае если она ещё не сформулирована, то тема проекта для тестирования выбирается из таблицы 3 стр. 21.

В выводах необходимо отразить всё новое, что вы узнали, выполняя лабораторную работу. Например, что представляет собой тест-план, чек-лист и тест-кейс. Перечислять, что вы сделали в самой лабораторной работе, в выводе нет необходимости.

Контрольные вопросы

1. Что такое тестирование программного обеспечения ?
2. Какие разделы включены в план тестирования?
3. Что такое чек-лист?
4. Что содержит ожидаемый результат тест-кейса?
5. Что относятся к атрибутам тест-кейса?

[6] Литература\TestCaseCalc

ЛАБОРАТОРНАЯ РАБОТА №2 Модульное тестирование, использование JUnit

Цель: познакомиться с модульным тестированием, которое является наиболее ранним видом тестирования выполняемом на стадии кодирования и выполняемого самими разработчиками.

Краткие теоретические сведения

Модульное тестирование (unit testing, юнит-тестирование) – вид тестирования, направленный на оценку корректности исходного кода программы. Модульные тесты покрывают атомарные участки кода, что позволяет удостовериться в их работоспособности (в т.ч. после внесения изменений). Модульные тесты проверяют как отдельные методы, так и взаимодействие объектов в проекте.

Преимущества модульных тестов

• Повышают качество архитектуры приложения.
• Стимулируют написание простых методов.
• Поощряют изменения в коде.
• Упрощают интеграцию кода.
• Помогают документированию кода.
• Минимизируют зависимости в системе.
• Создаются на основе бесплатных фреймворков.
• Созданы для многих языков программирования.

В лабораторной работе мы будем использовать JUnit – фреймворк модульного тестирования ПО на языке Java. Он имеет широкий набор расширений – таких как jMock, HtmlUnit и т.д. Адаптирован для других языков, таких как: PHP, C#, Python, Delphi, Perl, C++, JavaScript и т.д.

Порядок выполнения работы

1. На начальном этапе выполним задачу по тестированию уже готового кода, представленного в виде файла calculator-1.0.jar. В данной работе есть возможность выполнять тестирование в различных средах программирования языка Java. Выберете одну из сред программирования и откройте соответствующую инструкцию. Задание нужно сделать только в одной из сред программирования:

- Создание теста Junit в IntelliJ[7];
- Создание теста Junit в NetBeans[8];
- Создание теста Junit вEclips[9];

[7] Создание теста Junit в IntelliJ.pdf
[8] Создание теста Junit в NetBeans.pdf
[9] Создание теста Junit вEclips.pdf

Пример, описанный в инструкции, нужно сделать обязательно, чтобы увидеть правильные результаты.

2. На втором этапе протестируем с помощью JUnit один из методов библиотеки calculator-1.0.jar.

При решении данной задачи необходимо использовать такие методы тест-дизайна как классы эквивалентности и граничные условия[10]. Чтобы определиться с классами эквивалентности и граничными условиями в данной задаче проще всего использовать для методов с одним параметром числовые прямые разбитые на участки, а при наличии двух параметров метода двумерные числовые плоскости с параметрами метода, отложенного по осям. Например, для тестирования метода сложения a+b создадим числовую плоскость как показаную на рисунке 1.

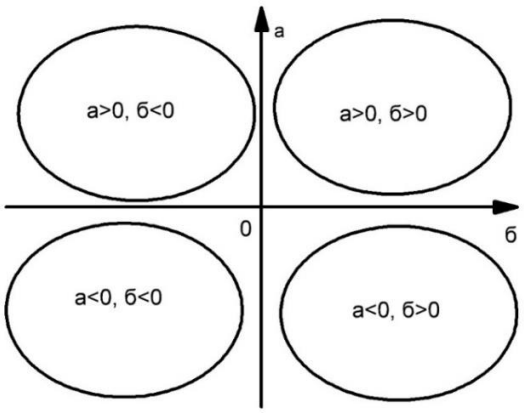


Рисунок 1 Графическое изображение классов эквивалентности для методов с двумя параметрами

Таким образом, мы получаем четыре класса эквивалентности и граничные условия, находящиеся на границе классов эквивалентности. Для максимального тестового покрытия данного метода необходимо создать четыре тестовых метода для проверки классов эквивалентности и пять тестовых методов для проверки граничных условий.

Задания выбрать согласно варианту из таблицы 1.

Таблица 1

№ варианта	**Тестируемый метод**	**№ варианта**	**Тестируемый метод**
1	sum(double, double)	9	pow(double, double)
2	sub(long, long)	10	sqrt(double)
3	sub(double, double)	11	tg(double)
4	mult(long, long)	12	ctg(double)
5	mult(double, double)	13	cos(double)
6	div(long, long)	14	sin(double)
7	isPositive(long)	15	isNegative(long)
8	div(double, double)		

[10] Куликов С.С. Тестирование программного обеспечения Базовый курс, раздел 2.7.2. «Классы эквивалентности и граничные условия» стр. 223

Содержание отчета

Разработать модульные тесты для тестирования библиотеки калькулятор с использованием классов эквивалентности и граничных условий.

В выводах необходимо отразить всё новое, что вы узнали, выполняя лабораторную работу. Например, что такое классы эквивалентности и граничные условия **приведя примеры из собственного варианта**. Перечислять, что вы сделали в самой лабораторной работе, в выводе нет необходимости.

Контрольные вопросы

1. Что такое классы эквивалентности?
2. Что такое модульные тесты?
3. Что проверяют модульные тесты?
4. Какие элементы подвергаются проверкам в рамках модульного тестирования?

ЛАБОРАТОРНАЯ РАБОТА №3 Разработка проекта на Java под управлением тестированием[11]

Цель: Научиться разрабатывать проект в стиле объектно-ориентированного программирования на примере языка Java.

Краткие теоретические сведения

Разработка под управлением тестированием (Test-Driven Development, TDD) – техника разработки ПО, опирающаяся на очень короткие повторяющиеся циклы, в которых написание тестов предшествует написанию кода.

Особенности модульных тестов

Модульные тесты всегда должны проходить на 100%, т.к. в ином случае у нас есть ТОЧНО неработоспособный код, что ГАРАНТИРОВАННО приводит к дефектам в приложении. ***Также*** повторное успешное выполнение 100% модульных тестов показывает, что в процессе доработок кода в нём не было ничего нарушено.

- Модульные тесты отделены от кода приложения на уровне структуры проекта.
- Модульные тесты независимы, отделены друг от друга, атомарны, просты.
- Модульные тесты часто пишутся программистами (т.к. часто требуется очень хорошо знать программирование, чтобы реализовать ту или иную логику).

[11] «Автоматизированное тестирование» Куликов С.С. раздел 4.4. Разработка под управлением тестированием стр. 165

- Модульные тесты могут писаться ДО кода приложения

В рамках модульного тестирования проверкам подвергаются:

- Метод.
- Метод, который ничего не возвращает.
- Класс.
- Взаимодействие классов.
- Геттеры и сеттеры.
- Конструкторы.
- Исключения.

Все эти варианты следует реализовать на данном этапе лабораторной работы.

Порядок выполнения работы

В качестве примера рассмотрим задачу[12]:

Дана окружность со следующими атрибутами: радиус, диаметр, длина окружности, площадь круга. Задано только одного значение этих элементов. Остальные элементы окружности

Разработать класс окружность

1) Для проектируемого класса выделим одно поле радиус
2) Конструктор, определяющий значение радиуса из заданного атрибута: радиус или диаметр или длина окружности или площадь круга;
3) Геттер и сеттер для радиуса;
4) Методы вычисления атрибута: радиус, диаметр, длина окружности, площадь круга;
5) Методы заполнения поля радиус при вводе нового атрибута: радиус или диаметр или длина окружности или площадь круга;

На третьем этапе необходимо протестировать самостоятельно разработанный проект согласно варианту из таблицы 2. Поскольку модульные тесты покрывают атомарные участки кода, которыми в ООП соответствуют методы то разработку тестируемого проекта необходимо разрабатывать в стиле ООП с использованием классов и методов (не менее 5 методов).

Задание:

Разработать проект на языка Java в стиле объектно-ориентированного программирования.

Задания выбрать согласно варианту из таблицы 2.

Таблица 2

№ варианта	**Задание для тестируемого проекта**
1	Дан номер года (положительное целое число). Определить количество дней в этом году, учитывая, что обычный год

[12] Пример рассмотрен в документе «Тестирование с использованием Maven»

	насчитывает 365 дней, а високосный — 366 дней. Високосным считается год, делящийся на 4, за исключением тех годов, которые делятся на 100 и не делятся на 400 (например, годы 300, 1300 и 1900 не являются високосными, а 1200 и 2000 — являются).
2	Даны целочисленные координаты точки на плоскости. Если точка совпадает с началом координат, то вывести 0. Если точка не совпадает с началом координат, но лежит на оси OX или OY, то вывести соответственно 1 или 2. Если точка не лежит на координатных осях, то вывести 3.
3	Даны три переменные вещественного типа: A, B, C. Вывести их значения в порядке возрастания.
4	Даны координаты точки, не лежащей на координатных осях OX и OY. Определить номер координатной четверти, в которой находится данная точка.
5	Дано целое число. Вывести его строку-описание вида «отрицательное четное число», «нулевое число», «положительное нечетное число» и т. д.
6	Арифметические действия над числами пронумерованы следующим образом: 1 — сложение, 2 — вычитание, 3 — умножение, 4 — деление. Дан номер действия N (целое число в диапазоне 1–4) и вещественные числа A и B (B не равно 0). Выполнить над числами указанное действие и вывести результат.
7	Элементы равнобедренного прямоугольного треугольника пронумерованы следующим образом: 1 — катет a, 2 — гипотенуза c = a·(2)1/2, 3 — высота h, опущенная на гипотенузу (h = c/2), 4 — площадь S = c·h/2. Дан номер одного из этих элементов и его значение. Вывести значения остальных элементов данного треугольника (в том же порядке).
8	Единицы длины пронумерованы следующим образом: 1 — дециметр, 2 — километр, 3 — метр, 4 — миллиметр, 5 — сантиметр. Дан номер единицы длины (целое число в диапазоне 1–5) и длина отрезка в этих единицах (вещественное число). Найти длину отрезка в метрах.
9	Даны целочисленные координаты точки на плоскости. Если точка совпадает с началом координат, то вывести на экран 0, если точка не совпадает с началом координат, но лежит на оси OX или OY, то вывести на экран 1. Если точка не лежит на координатных осях, то вывести на экран 2.
10	Локатор ориентирован на одну из сторон света (1 — север, 2 — запад, 3 — юг, 4 — восток) и может принимать три цифровые команды поворота: 1 — поворот налево, –1 — поворот направо, 2 — поворот на 180°. Дан символ C — исходная ориентация локатора и целые числа N1 и N2 — две посланные команды.

	Вывести ориентацию локатора после выполнения этих команд.
11	Даны целые числа a, b, c. Вывести на экран 0 если каждое из них отлично от другого, 1 если только два числа одинаковы и 3 если одинаковы все три числа.
12	Даны три вещественных числа: A, B, C. Если их значения упорядочены по возрастанию, то удвоить их; в противном случае заменить значение каждой переменной на противоположное по знаку. Вывести на экран новые значения переменных A, B, C.
13	Даны целые числа a, b, c, являющиеся сторонами некоторого треугольника. Вывести на экран заключение о том, каким является этот треугольник (равносторонним, равнобедренным или обычным).
14	Даны целые числа a, b, c. Вывести на экран 0 если каждое из них отлично от другого, 1 если только два числа одинаковы и 3 если одинаковы все три числа.

В процессе разработки проекта необходимо протестировать все его компоненты в рамках модульного тестирования.

Содержание отчета

Предоставить программы, написанные на языке Java, модульные тесты, а также результаты её работы программы и результаты тестирования.

В выводах необходимо отразить всё новое, что вы узнали, выполняя лабораторную работу. Например, что может подвергаться проверкам в рамках модульного тестирования, **приведя примеры из собственного варианта**.

Перечислять, что вы сделали в самой лабораторной работе, в выводе нет необходимости.

Контрольные вопросы

1. Что необходимо сделать, если в рамках модульного тестирования необходимо проверить действие, опирающееся на внешние источники или приёмники данных?
2. Нужно ли проверять, что операция создания экземпляра класса вернула экземпляр нужного класса?
3. Для чего используются аннотации в JUnit?
4. Как проверку assertEquals можно заменить проверкой assertTrue?
5. Имеет ли смысл проверять модульными тестами геттеры и сеттеры?

ЛАБОРАТОРНАЯ РАБОТА №4 Тестирование внешних зависимостей, использование jMock.

Цель: изучить возможности тестирования в модульных тестах внешних зависимостей используя для этого Mock-объекты.

Краткие теоретические сведения

Внешняя зависимость (external dependency) – нечто, что нам неподконтрольно в процессе выполнения тестов, например:

- база данных;
- веб-сервис;
- сторонние библиотеки.

Не имея возможности контролировать внешнюю зависимость, мы не можем гарантированно проверить поведение тестируемого приложения. Решение: «подменить» зависимость объектом, который мы можем контролировать



Как мы проверим запрос, не имея доступа к СУБД?



Создать специальный объект, у которого мы можем запросить **<u>любые</u>** данные.

Mock-объекты (англ. mock – подделка) – это объекты, которые имеют тот же интерфейс, что и компоненты приложения, но полностью управляемые из теста.

Их использование позволяет избежать использования полной инфраструктуры, необходимой приложению для запуска.

Что ещё более важно, можно проконтролировать, что код вызывал те или иные методы у mock-объекта с теми или иными аргументами.

Для реализации тестирования внешних зависимостей на языке Java используется фреймворк модульного тестирования ПО jMock, расширение JUnit.

Порядок выполнения работы

В качестве примера рассмотрим проект, разработанный в стиле ООП содержащий класс автомобиль.

```
public class Car {


    private String marka;
    private int year;
    private String owner;

    public Car(String marka, int year, String owner) {
        this. marka = marka;
        this.year = year;
        this.owner = owner;
    }

    public String getMarka () {
        return marka;
    }

    public int getYear() {
        return year;
    }

    private String requestOwnerToCar(String owner){
        //запрос к БД которая не существует
        String car=request(owner);
        return car;
    }
```

```
    public int calcPowerToSpped (int pow){
        //Запрос к ресурсу который вычисляет скорость авто
        int spped= request(pow);
        return spped;
    }
}
```

Создадим новый тестовый класс CarTest и напишем тест для проверки конструктора

```
public class CarTest {
 @Test
    void testCreateCar(){

        Car car = new Car("", 2014, "");// создать объект типа карт
        Car newCar = Mockito.mock(Car.class);
```

вызываем метод для эмуляции и указываем тот класс объект, какого типа мы хотим создать.

Теперь используя геттеры для получения значений полей объекта выполним проверку Для строк по умолчанию будет значение null, для целого числа 0.

```
        assertEquals(null, newCar. getMarka());
        assertEquals(0, newCar.getYear());
}
```

В следующем тесте сгенерируем на выходе метода не значение по умолчанию, а заданное нами значение. Для этого применим метод when.

```
@Test
    void remoteServissReturnValue(){
        Car newCar = Mockito.mock(Car.class);
        when(newCar. calcPowerToSpped (120)).thenReturn(250);
        assertEquals(120, newCar. calcPowerToSpped (250));
    }
```

```
@Test
void getOwner(){
    when(newCar.getOwner()).thenReturn("Mercedes");
    assertEquals("Mercedes", newCar.getOwner());
}
```

Данный тест мы используем, когда общаемся с удалённым сервисом.

Теперь проверим не только конечный результат, но и то как мы к нему пришли, то есть метод verify позволяет нам проверить вызывались ли какие-то конкретные методы нашего объекта.

```
@Test
void verificationTest(){
    //assertEquals(null, newCar. getMarka());
    String Marka= newCar. getMarka();
    verify(newCar, only()).getManufacturer();
}
```

Содержание отчета

Предоставить модульные тесты для тестирования внешних зависимостей. Объект для тестирования выбрать согласно варианту из таблицы 2 из лабораторной работы №2.

Выполнить проверку:

- Конструктора;
- Геттеров;
- Метода возвращающего заданное нами значение;
- Вызывались ли конкретные методы;

Каждую проверку оформить в отдельном тесте!!!

В выводах необходимо отразить всё новое, что вы узнали, выполняя лабораторную работу. Например, какие проверки вы использовали в Selenium IDE. Перечислять что вы сделали в самой лабораторной работе в выводе нет необходимости.

Контрольные вопросы

1. Какими рекомендуется создавать модульные тесты?

2. Можно ли с помощью jMock проэмулировать поведение класса, реализующего взаимодействие с внешней средой?
3. Можно ли проэмулировать некоторую сущность с помощью jMock, если уже написан реально существующий класс для этой сущности?
4. В каких условиях применять mock-обекты и писать модульные тесты проще?

ЛАБОРАТОРНАЯ РАБОТА №5 Тестирование web-приложений, Selenium IDE

Цель: изучить возможности функционального тестирования на примере автоматизированных тестов, разработанных с использованием Selenium IDE.

Краткие теоретические сведения

Функциональное тестирование программного обеспечения (ПО) представляет собой комплекс ключевых мероприятий по проверке программного обеспечения, по результатам которых устанавливается соответствие этого ПО исходным требованиям заказчика. Иными словами, с помощью проведения данных тестов устанавливается способность информационных систем в конкретных условиях решать пользовательские задачи.

Таким образом, **функциональное тестирование** это вид тестирования, направленный на проверку корректности работы функциональности приложения.

В рамках функционального тестирования программного продукта проверяются следующие компоненты и критерии:

- пользовательский интерфейс;
- API;
- базы данных;
- безопасность систем;
- работа клиентских и серверных приложений;
- общая функциональность.

Такое тестирование можно проводить как в ручном, так и в автоматическом режиме.

В лабораторной работе рассмотрим автоматизированное создание тестов с использованием Selenium IDE[13].

[13] Святослав Куликов «Автоматизированное тестирование Рабочая тетрадь» раздел 7.2.2. «Использование Selenium IDE» Стр: 252

Selenium IDE — это расширение для браузера, которое записывает и воспроизводит действия пользователя. Тесты записываются, как запись видео, а воспроизведение записанных тестов аналогично воспроизведению видео. Является библиотекой Selenium с графическим интерфейсом и возможностями для работы со сценариями тестирования веб-страниц. IDE генерирует код Selenium WebDriver, который повторяет записанные действия пользователей.

Порядок выполнения работы

Перечислим порядок действий для тестирования ресурса «isit.vstu.by»[14]

1. Запустим плагин Selenium IDE;
2. Создадим новый тест в новом проекте;
3. Зададим имя проекта «isit.vstu.by»;
4. Зададим базовый URL «https://isit.vstu.by/»;
5. Запустим запись теста;
6. Проверим, туда ли мы попали, проверяя заголовок страницы;
7. Протестируем поисковый сервис:
    a. Введем в строке поиска «Соколова»;
    b. Запустим поиск
    c. В открывшемся окне «Результаты поиска» найдем текст «Соколова Анна Сергеевна»;
8. Остановим запись и откажемся создавать новый тест;

В окне плагина Selenium IDE как показано на рисунке 2 представлены команды для тестирования[15], но прежде чем запустить тест на выполнение уберем лишние команды. Selenium IDE записывает любое действие, в том числе и некоторые, не нужные нам.

9. Закоментарим лишние команды, а позже их удалим;
10. Запустим тест;
11. С результатами прохождения теста можно ознакомиться на вкладке «Log»

14 Пример использования Selenium IDE приведен в видео «Selenium IDE JIPN2.mp4»
[15] Информацию о команде можно почитать на вкладке «Reference»

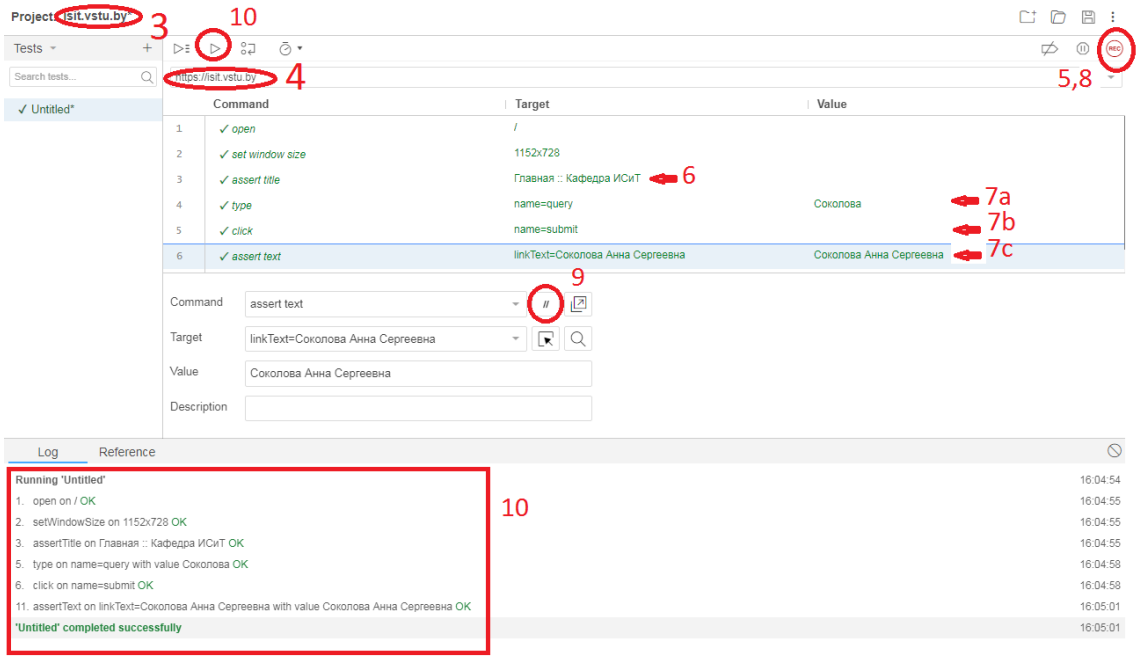


Рисунок 2 Окно плагина Selenium IDE

Объект для тестирования выбрать согласно варианту из таблицы 3.

Таблица 3

№ варианта	Объект для тестирования	№ варианта	Объект для тестирования
1	brsm.vstu.by	15	kitik.vstu.by
2	cit.vstu.by	16	kito.vstu.by
3	design.vstu.by	17	malp.vstu.by
4	economics.vstu.by	18	management.vstu.by
5	fl.vstu.by	19	matem.vstu.by
6	fil.vstu.by	20	mat-tech.vstu.by
7	fis.vstu.by	21	mk.vstu.by
8	fitr.vstu.by	22	otip.vstu.by
9	fl.vstu.by	23	ovr.vstu.by
10	forlang.vstu.by	24	sgd.vstu.by
11	fpt.vstu.by	25	smk.vstu.by
12	ic.vstu.by	26	smu.vstu.by
13	isap.vstu.by	27	stud.vstu.by
14	it.vstu.by	28	zf.vstu.by

Содержание отчета

Разработать функциональные тесты. Объект для тестирования выбрать согласно варианту из таблицы 3.

Выполнить проверку:

- Работы ссылок;
- Полей ввода и поиска;

- Наличие элементов на странице;

Каждую проверку оформить в отдельном тесте!!!

Количество тестов не менее десяти.

В выводах необходимо отразить всё новое, что вы узнали, выполняя лабораторную работу. Например, какие проверки вы использовали в Selenium IDE. Перечислять что вы сделали в самой лабораторной работе в выводе нет необходимости.

Контрольные вопросы

1. Как работает Selenium IDE?
2. Что можно записать с помощью технологии Record & Playback в Selenium IDE?
3. В каком поле записывается указание на объект, с которым выполняется действие в Selenium IDE?
4. В чем заключается разница между проверками assert и verify в Selenium IDE?

ЛАБОРАТОРНАЯ РАБОТА №6 Автоматизированное тестирование Selenium WebDriver на языке Java

Цель: изучить возможности функционального тестирования на примере автоматизированных тестов, разработанных с использованием Selenium WebDriver.

Краткие теоретические сведения

Selenium WebDriver - популярный инструмент для управления реальным браузером, который можно использовать как для автоматизации тестирования веб приложений, так и для выполнения других рутинных задач, связанных с работой в вебе.

Это фреймворк для управления браузерами, основной продукт комплекта Selenium. Представляет из себя семейство драйверов для разных браузеров (Firefox, Edge, Google Chrome/Chromium, Internet Explorer, Safari, Opera) и набор клиентских библиотек на разных языках программирования для работы с драйверами. WebDriver поддерживает работу с языками Java, .Net (C#), Python, Ruby, JavaScript.

WebDriver напрямую отправляет команды браузеру, используя его API и получает результаты тестирования. WebDriver же использует способ взаимодействия с браузером, максимально близкий к действиям обычного пользователя.

Поскольку Selenium WebDriver представляет собой фреймворк, для его использования нужен язык программирования, на котором будем писать

тесты и среда разработки для этого языка. В качестве языка программирования выберем язык Java, а среду разработки будем использовать IntelliJ IDE.

Порядок выполнения работы

Первая часть работы будет состоять в управлении браузером Google Chrome[16].

Решение этой задачи будет состоять из следующих шагов:

1. Создадим консольный проект;
2. Подключим необходимый фреймворк (selenium-server-4.15.0.jar);
3. Текст программы возьмём из файла, полученного в результате экспорта из Selenium IDE.
4. Из файла экспорта возьмём только команды открытия браузера и вывод заголовка страницы на консоль.

Листинг программы управления браузером Google Chrome:

```
import org.openqa.selenium.chrome.ChromeDriver; //драйвер браузера Google Chrome
import org.openqa.selenium.WebDriver;// экземпляр браузера Google Chrome

public class Main {
   public static void main(String[] args) {
      WebDriver driver;
      driver = new ChromeDriver();// создаем экземпляр браузера
      driver.get("https://isit.vstu.by/");// открываем заданную страницу
      System.out.println(driver.getTitle()); //выводим ев консоль заголовок
   }
}
```

Данная задача доказала, что мы можем управлять браузером!!!

[16] Пример работы представлен в документе Автотест Selenium Webdriver на Java.pdf

Содержание отчета

Предоставить приложение, выполняющее с помощью фреймворка Selenium WebDriver открытие в браузере веб-страницы согласно варианту (смотри таблицу 3) и вывести в консоль её заголовок.

Предоставить приложение, выполняющее функциональные автотесты с использованием библиотеки JUnion и фреймворка Selenium WebDriver.

Решение этой задачи должно состоять из следующих шагов:

1. Создадим консольный проект;
2. Подключим необходимый (selenium-server-4.15.0.jar, junit-4.12.jar, hamcrest-core-1.3.jar);
3. Текст программы возьмём из файлов, полученных в результате экспорта из Selenium IDE.
4. Берем только те строки, которые будут задействованы в том или ином тесте. В нашем случае это проверка поиска и проверка ссылки сотрудники.

Листинг тестового класса для функциональных автотестов проверки поиска и ссылки.[17]

```
public class isitTest {
   private WebDriver driver;

@BeforeClass // запускается только один раз при запуске класса
   public void setUpClass() {
      driver = new ChromeDriver();
      driver.get("https://isit.vstu.by/");
   }
   @AfterClass // запускается после того, как отработали все тестовые методы.
      public void tearDownClass() {
         driver.quit();
```

[17] Более подробно об использовании JUnit можно прочитать «Учебник JUnit»

```
    @Test
    public void searchTest() {
        assertThat(driver.getTitle(), is("Главная :: Кафедра ИСиТ"));// проверка заголовка страницы
        driver.findElement(By.name("query")).sendKeys("Соколова");//ввод строки поиска
        driver.findElement(By.name("submit")).click();//нажатие кнопки поиск
        assertThat(driver.getTitle(), is("Результаты поиска :: Кафедра ИСиТ")); //проверка заголовка страницы
        assertThat(driver.findElement(By.linkText("Соколова Анна Сергеевна")).getText(), is("Соколова Анна Сергеевна")); //проверка текста
    }
    @Test
    public void linkKafedraTest() {
        assertThat(driver.getTitle(), is("Главная :: Кафедра ИСиТ")); // проверка заголовка страницы
        driver.findElement(By.linkText("Сотрудники")).click(); //нажатие ссылки
        assertThat(driver.findElement(By.cssSelector(".container:nth-child(5) .ccm-block-testimonial-name")).getText(), is("Соколова Анна Сергеевна")); //проверка текста
    }
}
```

Выполнить экспорт тестов, разработанных в лабораторной работе №4. выполняющие проверки:

- Ссылок;
- Полей ввода и поиска;
- Наличие элементов на странице;

В количестве не менее десяти тестов. Также необходимо разработать тестовый класс для проверки веб-ресурса согласно варианту таблицы 3.

Каждую проверку оформить в отдельном тесте!!!

В выводах необходимо отразить всё новое, что вы узнали, выполняя лабораторную работу. Например, как выполняется поиск элементов на странице какие методы используются для проверки и т.д. Перечислять что вы сделали в самой лабораторной работе в выводе нет необходимости.

Контрольные вопросы

1. Как перенести тесты из Selenium IDE в Selenium Web Driver?
2. Можно ли с помощью Selenium Web Driver провести тестирование приложения без использования настоящего браузера?
3. Что рекомендуется использовать в качестве-локаторов в Selenium Web Driver?
4. Позволяет ли Selenium Web Driver выполнять тесты Selenium IDE?
5. Гарантирует ли Selenium Web Driver то, что тесты будут одинаково выполняться в разных браузерах?

ЛАБОРАТОРНАЯ РАБОТА №7 Автоматизированное тестирование Selenium WebDriver на языке C#

Цель: изучить возможности функционального тестирования на примере автоматизированных тестов, разработанных с использованием Selenium WebDriver.

Краткие теоретические сведения

Фреймворк Selenium WebDriver поддерживает разные языки программирования такие как Java, C#, PHP, Ruby, Perl, Python.

C# — объектно-ориентированный язык программирования, созданный на основе C++ и Java. C# позволяет разработчикам создавать приложения с использованием Visual Studio на платформе .Net.

Visual Studio — это интегрированная среда разработки (IDE), которая используется разработчиками для создания приложений на различных платформах, таких как Windows, Android, iOS и облачные приложения.

NUnit - это Модульное тестирование фреймворк, поддерживаемый Visual Studio и Selenium WebDriver. NUnit — наиболее широко используемый фреймворк модульного тестирования для приложений .Net. NUnit представляет результаты тестирования в удобном для чтения формате и позволяет тестировщику отлаживать автоматизированные тесты.

В данной лабораторной работе создадим функциональные тесты на языке программирования C#, а в качестве среды разработки будем использовать Microsoft Visual Studio.

Интеграция Selenium с платформой NUnit позволяет тестировщику различать различные классы тестов. NUnit также позволяет тестировщикам использовать аннотации, такие как SetUp, Test и TearDown, для выполнения действий до и после запуска теста.

Фреймворк NUnit можно интегрировать с Selenium создав тестовый класс NUnit и запустив его с использованием платформы NUnit.

Порядок выполнения работы

Работа будет состоять в управлении браузером FireFox [18]. Решение этой задачи будет состоять из следующих шагов:

1. Создадим консольное приложение (.NET Framework);
2. Текст программы возьмём из файла, полученного в результате экспорта из Selenium IDE.

Содержание отчета

Выполнить экспорт тестов, разработанных в лабораторной работе №4. выполняющие проверки:

- Ссылок;
- Полей ввода и поиска;
- Наличие элементов на странице;

В количестве не менее десяти тестов. Также необходимо разработать тестовый класс для проверки веб-ресурса согласно варианту таблицы 3 стр. 21.

Каждую проверку оформить в отдельном тесте!!!

В выводах необходимо отразить всё новое, что вы узнали, выполняя лабораторную работу. Например, какие отличия в тестах, написанных на Java и на C#. Перечислять что вы сделали в самой лабораторной работе в выводе нет необходимости.

Контрольные вопросы

1. Как и для чего нужна конструкция using?
2. Какие библиотеки нужны для модульного тестирования на C#?
3. Какие команды используются для проверок в модульных тестах?
4. Назовите порядок команд перехода по ссылкам.

[18] Пример работы представлен в документе Автотест Selenium WebDriver на C.pdf

ЛАБОРАТОРНАЯ РАБОТА №8 Поиск и документирование дефектов
Цель: научиться документировать найденные дефекты.

Краткие теоретические сведения

Дефекты, обнаруженные в процессе тестирования, должны быть корректно и понятно документированы, так чтобы разработчик мог быстро воспроизвести и устранить данный дефект. Описание дефекта (баг-репорт) можно сохранить в специализированной багтрэкинговой системе (например, JIRA, Bugzilla, Mantis, Redmine и др.) или в табличном виде[19].

Упрощённые определения:

Дефект – расхождение ожидаемого и фактического результата.

Ожидаемый результат – поведение системы, описанное в требованиях.

Фактический результат – поведение системы, наблюдаемое в процессе тестирования.

Дефекты могут встречаться в любой документации, в архитектуре и дизайне, в коде программы и т.д.

Отчёт о дефекте – документ, описывающий и определяющий важность обнаруженного дефекта, а также содействующий его устранению.

Цели создания отчёта о дефекте:

• предоставить информацию о проблеме;

• определить важность проблемы;

• содействовать устранению проблемы.

Основная цель написания отчёта о дефекте – устранение дефекта. Поэтому стоит помнить, что хороший тестировщик – не тот, кто написал за день 1000 бесполезных и бессмысленных отчётов, а тот, по чьим отчётам (вне зависимости от их количества) было исправлено большое количество дефектов.

Универсальная формула отчёта о дефекте состоит из трёх простых пунктов:

1. Что сделали (шаги воспроизведения).
2. Что получили (фактический результат).
3. Что ожидали получить (ожидаемый результат).

Формула «Что сделали? Что получили? Что ожидали?» хороша по многим причинам:

• Прозрачность: нельзя отклониться в повествовательный стиль и/или написать что-то, отличное от отчёта о дефекте.

• Легко верифицировать дефект.

[19] Куликов С.С. Тестирование программного обеспечения Базовый курс, раздел 2.5.3. «Атрибуты (поля) отчёта о дефекте» стр. 169

• Чётко (ещё до воспроизведения) видно, является ли описанное дефектом.

• Избавление от лишней бессмысленной коммуникации.

• Этой форме легко обучить «несмышлёных пользователей»

Порядок выполнения работы

В качестве примера составим баг-репорт описывающий дефект, позволяющий выполнить регистрацию при загрузке в поле "Фотография" файлов произвольного формата (exe, zip и другие).

- **Идентификатор**: уникальное значение с осмысленным компонентом – megaform_1;
- **Шаги по воспроизведению:** описывают действия, которые необходимо выполнить для воспроизведения дефекта.
  1. Открыть страницу http://svyatoslav.biz/testlab/megaform.php;
  2. Заполните все необходимые поля;
  3. Загрузить файл с расширением exe в поле «Фотография»;
  4. Нажать кнопку «Регистрация»;

  Дефект: ожидалось, что система отреагирует на файл, не содержащий фотографию сообщением или ошибкой;

- **Подробное описание:** информация о дефекте в развёрнутом виде – Когда все необходимые поля в форме регистрации заполнены, есть возможность загружать файлы разного формата (exe, zip и другие) в поле «Фотография». При этом регистрация проходит успешно.
  Ожидаемый результат: сообщение или ошибка;
  Фактический результат: загрузка файлов;
  Требование: -
- **Краткое описание:** состоит из ответов на вопросы «Что произошло?» «Где это произошло»? «При каких условиях это произошло?» - регистрация проходит успешно, когда в поле "Фотография" формы есть возможность загружать файлы разного формата.
- **Воспроизводимость:** показывает, при каждом ли прохождении по шагам воспроизведения дефекта удаётся вызвать его проявление – всегда;
- **Важность:** показывает степень ущерба, который наносится проекту существованием дефекта – высокая;
- **Срочность**: показывает, как быстро дефект должен быть устранён – обычная;
- **Симптом**: позволяет классифицировать дефекты по их типичному проявлению. - Неожиданное поведение;
- **Возможность обойти:** показывает, существует ли альтернативная последовательность действий – нет;
- **Комментарий:** может содержать любые полезные для понимания - Это рискованно для безопасности и бизнеса, как упущенная выгода.

- **Приложения:** список прикреплённых к отчёту о дефекте приложений - C:\Temp\ Screenshot 2024-09-26.png;

Результаты работы сведем в таблицу4.

Отчёт о дефекте Таблица 4

Атрибут отчёта о дефекте	Содержание
Идентификатор	megaform_1
Краткое описание	Регистрация проходит успешно, когда в поле "Фотография" формы есть возможность загружать файлы разного формата.
Подробное описание	Когда все необходимые поля в форме регистрации заполнены, есть возможность загружать файлы разного формата (exe, zip и другие) в поле «Фотография». При этом регистрация проходит успешно. **Ожидаемый результат:** сообщение или ошибка; **Фактический результат:** загрузка файлов; **Требование: -**
Шаги по воспроизведению	1. Открыть страницу http://svyatoslav.biz/testlab/megaform.php; 2. Заполните все необходимые поля; 3. Загрузить файл с расширением exe в поле «Фотография»; 4. Нажать кнопку «Регистрация»; **Дефект**: ожидалось, что система отреагирует на файл, не содержащий фотографию сообщением или ошибкой;
Воспроизводимость	Всегда
Важность	Высокая
Срочность	Обычная
Симптом	Неожиданное поведение
Возможность обойти	Нет;

Комментарий	Это рискованно для безопасности и бизнеса, как упущенная выгода.
Приложения	C:\Temp\ Screenshot 2024-09-26.png;

Содержание отчета

Оформить отчеты о дефектах, найденные на указанном преподавателем ресурсе. Составить для них отчет в форме таблицы. Количество описанных дефектов не менее 5.

В выводах необходимо отразить всё новое, что вы узнали, выполняя лабораторную работу. Например, что порядок заполнения полей отчета о дефектах не совпадает с последовательностью из заполнения. Перечислять что вы сделали в самой лабораторной работе в выводе нет необходимости.

Контрольные вопросы

1. Что такое дефект?
2. Для чего служит отчёт о дефекте?
3. В каком поле отчёта о дефекте располагается ответ на вопросы "Что? Где? При каких условиях?"?
4. Что такое важность и срочность дефекта?
5. Что и где нужно указать в отчёте о дефекте, если некорректная операция не позволяет продолжить дальнейшее тестирование?

ЛИТЕРАТУРА

1. Куликов С.С. Тестирование программного обеспечения Базовый курс Версия книги 1.0.3 от 23.09.2015
2. Святослав Куликов Автоматизированное тестирование Рабочая тетрадь © EPAM Systems, RD Dep., 2016
3. Книга по Java 1 ????
4. Книга по Java 2 ????
5. Культин, Н. Б. Основы программирования в Microsoft Visual C# / Н. Б. Культин. – Санкт-Петербург: БХВ, 2011. – 368 с.
6. Windows Forms C# – уроки создания приложения на itProger / [Электронный ресурс] // Сообщество программистов и онлайн-школа IT профессий itProger. – Режим доступа: https://itproger.com/course/csharp-app. – Дата доступа: 17.04.2024.